FACULTY OF ARTS

KU LEUVEN

Scripting languages

Lecture 5: Files, paths & modules

Tim Van de Cruys



Wednesday 21 October 2026

Working with files

- Up till now, we applied the bits of code we wrote to toy data
- In real applications, we usually want to apply our code to data stored on the computer (and then save the processed data as well)
- Today: the tools around the code
  - **Find** the right files: directories, paths, the command line
  - **Read** data from disk and **write** results
  - **Reuse** code: modules

Files and directories

- Data is stored in **files** as a sequence of bytes
- Files can be stored in **directories** (folders)
- Directories can contain files and other directories
  - The file system is a tree structure
  - The location of a file is a **path** through this tree
- Directories are useful to organize your files
- No need to put everything in one big pile
- Example: separate code, data, results

Paths

- A **path** describes the location of a file
  - In the simplest case, just the filename is enough to refer to a file in the same directory as your program/notebook:
    - `example.txt`
  - If a file is somewhere else, need to include directories:
    - `data/example.txt`
    - `data/novels/19thcentury/austen/emma.txt`
  - Use '..' (two dots) to refer to the parent directory
- NB: Windows uses \ instead of /
- However, this causes problems: use / in Python

Absolute and relative paths

- An **absolute** path starts at the root of the file system
  - `/home/tim/project/data/emma.txt` (Linux)
  - `/Users/tim/project/data/emma.txt` (Mac)
  - `C:/Users/tim/project/data/emma.txt` (Windows)
- A **relative** path starts from the **working directory**: the directory your program or notebook is running in
  - `data/emma.txt`
  - `../data/emma.txt`
- Absolute paths only work on your computer; relative paths work for everyone who has the same directory structure
  - Prefer relative paths in your code

Where am I?

- A relative path is only meaningful if you know your working directory
- Ask Python:

```
In: import os
In: os.getcwd()
Out: '/home/tim/project/notebooks'
```

- For a notebook, the working directory is normally the directory the notebook is in
- `FileNotFoundError` usually means: the path is correct, but relative to a different directory

Try it: paths

```
project/
    notebooks/
        analysis.ipynb
    data/
        novels/
            emma.txt
    results/
```

- The notebook `analysis.ipynb` is running in `notebooks/`
- Which path do you use to read `emma.txt`?
- Which path do you use to write `counts.txt` into `results/`?

Anatomy of a path

A path like `data/novels/austen/emma.txt` has several parts:

Directory	`data/novels/austen/`
Filename	`emma.txt`
Base name	`emma`
Extension	`.txt`

- We often need to extract or manipulate these parts
- E.g. to sort files by name, or to construct an output filename from an input filename
- Python provides the `os.path` module for this

Manipulating paths with `os.path`

```
In: from os.path import basename, splitext, join, exists

In: basename('data/novels/austen/emma.txt')
Out: 'emma.txt'
In: splitext('emma.txt')
Out: ('emma', '.txt')
In: join('data', 'novels', 'emma.txt')
Out: 'data/novels/emma.txt'
In: exists('data/novels/emma.txt')
Out: True
```

- `basename`: removes the directory part
- `splitext`: splits into name and extension
- `join`: builds a path (with the right separator for your system)
- `exists`: check before you open

Reading a file

```
with open('document.txt') as in_file:
    for line in in_file:
        print(line)
```

- `with` statement automatically closes your file when you're done
- We iterate over the file line by line
- Each `line` is a string (including the newline character `'\n'`)
- Use `line.strip()` to remove leading/trailing whitespace and newlines
- `in_file.read()` reads the entire file as a single string

Reading and writing

```
with open('document.txt') as in_file:
    with open('processed_document.txt', 'w') as out_file:
        for line in in_file:
            line = line.replace('a', 'b')
            out_file.write(line)
```

- Second argument to `open`: the **mode**
  - `'r'`: reading (default)
  - `'w'`: writing (overwrites an existing file!)
  - `'a'`: appending
  - `'b'`: binary data (default is text)
- `write` takes one string and does not add a newline: add `'\n'` yourself
- Note: we can only read and process files in plain text format (`'txt'`)
- Loading something like Word or PDF files will not work because those are encoded in a binary format

Text encoding

- A text file is represented using a particular text encoding
- Particularly important for non-Western characters, or e.g. emojis
- You need to use the correct encoding when reading and saving your text

```
Fran�oise
Å-Ring
ßeta or Beta?
Öh löök, umlauts!
ENCYCLOPÆDIA
ça va! mon élève mi niña?
A stealthy fart
Đe lónlí blú bojs
```

```
Fran�oise
�-Ring
�eta or Beta?
�h l��k, umlauts!
ENCYCLOP�DIA
�a va! mon �l�ve mi ni�a?
A stealthy �art
�e l�nl� bl� bojs
```

```
FranÃ§oise
Ã…-Ring
Ãßeta or Beta?
Ã-h lÃ¶Ã¶k, umlauts!
ENCYCLOPÃ†DIA
Ã§a va! mon Ã©lÃ¨ve mi niÃ±a?
A stealthy Åfart
Ãe lÃ³nlÃ blÃº bojs
```

Text encoding

- Always use the Unicode character set with UTF-8 encoding
- Which is the default for Python (and many other programs)
- Might be more problematic with older datasets or Windows programs

Text encoding: specifying the encoding

- If you try to load a text in a different encoding, Python will raise an error
- Solution: specify the encoding explicitly

```
# This will cause an error if the file
# is not in UTF-8:
with open('document_latin1.txt') as in_file:
    for line in in_file:
        print(line)
# UnicodeDecodeError: 'utf-8' codec can't
# decode byte 0xe9 ...
```

```
# Fix: specify the encoding
with open('document_latin1.txt', encoding='latin1') as in_file:
    for line in in_file:
        print(line)
```

The command line

- Textual interface that takes commands for the computer's operating system as lines of text
- Also called: terminal, shell, console
  - Linux and Mac: Terminal (unix shell)
  - Windows: Anaconda Prompt or PowerShell (different commands)
- Useful for file management, and to run programs (e.g. `python`)
- Many tools in research only exist as command line tools
- Note: the command line is not Python; it is a separate language

Basic commands

	unix shell	Windows
print working directory	`pwd`	`cd`
list directory	`ls`	`dir`
change directory	`cd data`	`cd data`
go to parent directory	`cd ..`	`cd ..`
make directory	`mkdir results`	`mkdir results`
copy, move/rename, remove	`cp, mv, rm`	`copy, move, del`
show first lines of a file	`head emma.txt`	`more emma.txt`
run a Python script	`python script.py`	`python script.py`

- Paths work exactly as in Python: relative to the working directory, `..` for the parent
- Tab completes file names; the up arrow repeats the previous command

The command line from a notebook

- In a notebook, prepend an exclamation mark (`!`) to run a command on the command line

```
!ls data
!head data/emma.txt
!wget http://example.org/corpus.zip
!unzip corpus.zip
```

- Each `!` command runs in a fresh shell: `!cd data` has no lasting effect
- To change the working directory of the notebook itself: `%cd data`
- Jupyter commands that start with % are called **magic commands**

Try it: the command line

```
project/
    notebooks/
        analysis.ipynb
    data/
        novels/
            emma.txt
    results/
```

- You open a terminal in `project/`. Which commands do you type to:
  - list the files in `novels/`
  - move into `notebooks/`
  - from there, show the first lines of `emma.txt`

From files to tools

- We've already used `os.path`, and we'll soon use `glob` and `pickle`
- These are all examples of **modules**: reusable collections of code
- Modules allow us to go beyond Python's builtin functionality
- Let's look at how importing works

Importing modules

To go beyond builtin functionality, use import:

```
# import whole module:
import <module>
# or import a specific part:
from <module> import <function>
```

Importing modules

To go beyond builtin functionality, use import:

```
# import whole module:
import <module>
# or import a specific part:
from <module> import <function>
```

```
In: import math
In: math.pi
Out: 3.141592653589793
In: from math import sqrt
In: sqrt(16)
4.0
```

Matter of preference/convention which you use:

- `math.pi`: clear that pi comes from math module
- `sqrt`: more convenient usage

The Python standard library

- 'Batteries included'
- Many different modules with useful functions
- See `https://docs.python.org/3/library/index.html`
- Today:
  - `os.path`: manipulate paths
  - `glob`: find files by pattern
  - `pickle`: save Python objects to disk

Finding files by pattern

- Before we can read files, we often need to **find** them
- globbing: matching a set of files with a search pattern
- The most common operators are:
  - `*` meaning “match zero or more characters”
  - `?` meaning “match exactly one character”

Finding files by pattern

- Before we can read files, we often need to **find** them
- globbing: matching a set of files with a search pattern
- The most common operators are:
  - `*` meaning “match zero or more characters”
  - `?` meaning “match exactly one character”

Example:

```
In: from glob import glob
In: glob('scripting_practical?.ipynb')
Out: ['scripting_practical1.ipynb',
    'scripting_practical2.ipynb',
    'scripting_practical3.ipynb',]
In: glob('data/*.txt')
Out: ['data/austen.txt', 'data/dickens.txt', 'data/joyce.txt']
```

- The same patterns work on the command line: `ls data/*.txt`

Processing multiple files

```
from glob import glob

for filename in glob('data/*.txt'):
    with open(filename) as in_file:
        text = in_file.read()
        word_count = len(text.split())
        print(f'{filename}: {word_count} words')
```

- This is a common pattern: find files, then process them one by one
- `glob` returns the files in arbitrary order: use `sorted(glob(...))` if the order matters

Sorting with a key function

```
In: names = ['Charlie', 'Alice', 'Bob']
In: sorted(names)
Out: ['Alice', 'Bob', 'Charlie']
```

Sorting with a key function

```
In: names = ['Charlie', 'Alice', 'Bob']
In: sorted(names)
Out: ['Alice', 'Bob', 'Charlie']
```

```
# Sort by length instead of alphabetically:
In: sorted(names, key=len)
Out: ['Bob', 'Alice', 'Charlie']
```

Sorting with a key function

```
In: names = ['Charlie', 'Alice', 'Bob']
In: sorted(names)
Out: ['Alice', 'Bob', 'Charlie']
```

```
# Sort by length instead of alphabetically:
In: sorted(names, key=len)
Out: ['Bob', 'Alice', 'Charlie']
```

- The `key` argument: a function that is applied to each item *before* comparing
- `key=len`: sort by the length of each string
- Any function will do, also your own: `sorted(filenames, key=get_year)`
- `reverse=True`: sort from high to low

Saving Python objects with pickle

- What if we want to save a Python object (e.g. a dictionary or a list)?
- `pickle`: module for saving Python objects directly to disk

```
translation_dictionary = {'red': 'rouge', 'green': 'vert'}

import pickle
with open('translations.pickle', 'wb') as out_file:
    pickle.dump(translation_dictionary, out_file)

with open('translations.pickle', 'rb') as in_file:
    my_translations = pickle.load(in_file)

my_translations
Out: {'red': 'rouge', 'green': 'vert'}
```

- Binary mode (`'wb'`, `'rb'`): not human-readable, but preserves the exact Python object; useful to save intermediate results

The Python data science ecosystem

Jupyter	interactive notebooks
Spacy	Natural Language processing (NLP)
NLTK	Natural language processing (education)
Matplotlib	general plots/visualization
Seaborn	statistical data visualization
Pandas	tabular data analysis
Scikit-learn	Machine learning: classification, clustering
gensim	Topic modeling (LDA)

Installing packages

- Most of these are part of Anaconda distribution; already installed!
  - (Exceptions: Spacy, gensim)
- Install conda packages with Anaconda Navigator
- Install other packages with `pip`
  (more advanced, much more packages)
- Note: `pip` is a command line tool; in notebook, you can execute it with:
  - `%pip install spacy`

Practical session / reading

- Practical session 4
  - Notebook online
- The code-along notebook of this lecture is on Toledo: re-run the examples at home
- Read chapters 9 & 10 of Python for Everybody
  - We've now seen all the material of chapter 1-10, which covers the basics of Python

Project

- only for students taking the **6 ECTS** version of the course (e.g. Master of Digital Humanities, or Master of Sociology). Not required for students taking the 4 ECTS version of the course (e.g. Master of Artificial Intelligence).
- Choice between different options:
  - Exploratory data analysis of word ratings (datasets provided)
  - Exploratory analysis of your own data
  - Applications of programming techniques we discussed to your own research question (e.g. construction and analysis of your own corpus, analysis of a dataset of paintings, analysis of traffic streams data, ...)
- Deadline: early January
- Note: project is integral part for 6 ECTS; no submission will result in **NA** score

Grade calculation

- For 4 ECTS:
  - Final exam: 20/20
- For 6 ECTS
  - 4 ECTS part counts for 2/3
  - Project counts for 1/3
- Example (6 ECTS):
  - 12/20 (exam)
  - 15/20 (project)
  - $= 12 \times \frac{2}{3} + 15 \times \frac{1}{3} = 13/20$